VIETNAM NATIONAL UNIVERSITY, HO CHI MINH CITY
UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND ENGINEERING



DEEP LEARNING IN COMPUTER VISON

Exercise E1–E3

Advisor: Dr. Le Thanh Sach
Dr. Nguyen Quoc Minh

Student: Nguyễn Thị Linh Trang - 2570344
Nguyễn Lê Hoàng Hưng - 2570214

Ho Chi Minh City, November 2026

Contents

1 E1 — Image Classification: Softmax, MLP, CNN **2**
1.1 Training . . . 2
1.2 Architecture . . . 3
1.3 Results (test, seed 0) . . . 3
1.4 Remarks . . . 3
1.5 Error Analysis . . . 4
1.6 Conclusion . . . 6

2 E2 — Transformer: MSA and Tokenization **7**
2.1 Overview . . . 7
2.2 Multi-Head Self-Attention . . . 7
2.3 Tokenization . . . 8
2.4 Architecture and Training . . . 8
2.5 Results (test, seed 0) . . . 9
2.6 Remarks . . . 9
2.7 Error Analysis . . . 11
2.8 Conclusion . . . 12

3 E3 — LSTM / GRU on Image Sequences **13**
3.1 Training . . . 13
3.2 Converting Image to Sequence . . . 13
3.3 Architecture . . . 13
3.4 Results (test, seed 0) . . . 13
3.5 Remarks . . . 14
3.6 Conclusion . . . 15

General Information

Dataset: Fashion-MNIST (28×28 grayscale images, 10 classes), shared across E1–E3.

0.0.0.1 Choosing Fashion-MNIST over MNIST and CIFAR-10:

- **Compared to MNIST:** The MNIST dataset (consisting of handwritten digits from 0–9) is "over-saturated." Models like MLP and CNN can easily achieve around 98–99% accuracy on this dataset. Therefore, if MNIST were used, comparisons regarding architecture or representation methods would be compressed into a very small margin (only a few percent), making the differences between models difficult to observe.
- **Strengths of Fashion-MNIST:** The dataset provides an appropriate level of complexity that helps clearly distinguish the performance between models.
  - In exercise E1, Fashion-MNIST demonstrates a clear gap in accuracy among Softmax, MLP, and CNN.
  - In exercise E2, it allows for comparing the effectiveness of different tokenization methods.
  - In exercise E3, it creates conditions to compare the performance between RNN (LSTM/GRU) and CNN.

0.0.0.2 Data Splitting:

60,000 training images → 55,000 train / 5,000 validation (fixed by seed), 10,000 test images used only once at the end.

0.0.0.3 Preprocessing:

Divide by 255, normalize with mean 0.2860 / std 0.3530 (based on training set statistics). No data augmentation is used to purely compare architectures.

0.0.0.4 Reproducibility:

Seed 0 (determines the split, initialization, and batch order). Executed on CPU or GPU via Colab.

---

1 E1 — Image Classification: Softmax, MLP, CNN

1.1 Training

Data and split: See the general section at the beginning of the report. Custom training loop written from scratch (`forward` → `loss` → `backward()` → `optimizer.step()`), using cross-entropy loss (log-softmax + NLL).

Configuration for each model:
Full data logs are available in ***`outputs/train.log`****.*

Table 1: Training configuration for Softmax, MLP, and CNN.

Model	Optimizer	LR	Batch	Epoch	Regularization
Softmax	Adam	1e-2	128	10	–
MLP	Adam	1e-3	128	10	Dropout 0.2
CNN	Adam	1e-3	128	10	BatchNorm, Dropout 0.2

1.2 Architecture

Table 2: Architecture and parameter count for Softmax, MLP, and CNN.

Model	Architecture	Parameters
Softmax	Flatten(784) → Linear(784→10) → softmax	7,850
MLP	Flatten(784) → Linear(256) → ReLU → Dropout 0.2 → Linear(10)	203,530
CNN	2 × [Conv3×3 - BatchNorm - ReLU - MaxPool2] (32, 64 channels, 28→14→7) → Flatten(3136) → Dropout 0.2 → Linear(10)	50,282

1.3 Results (test, seed 0)

Table 3: Test results for Softmax, MLP, and CNN (seed 0).

Model	Parameters	Best Val Acc	Test Acc	Test Loss	Train Time
Softmax	7,850	83.36%	82.03%	0.520	6 s
MLP	203,530	89.04%	88.86%	0.316	10 s
CNN	50,282	92.12%	91.00%	0.255	192 s

1.4 Remarks

- **Capacity vs. Accuracy:** Softmax is a linear model, so it is capped at around 82%. The model cannot represent non-linear boundaries between classes, by adding a non-linear hidden layer (MLP), the accuracy increases by 6.64% (from 82.03% to 88.67%).

  Meanwhile, CNN has only 50,282 parameters, about 1/4 of the MLP, yet its accuracy is 2.33% higher (91.00% compared to 88.67%). Thanks to the weight sharing of filters and pooling, CNN exploits the 2D structure (local connectivity and translational invariance) that the flatten operation destroys. Therefore, accuracy does not increase proportionally with the number of parameters, but rather depends on whether the architecture fits the data format. However, CNN is slower than MLP on the CPU despite having fewer parameters, because convolutions are more time-consuming.

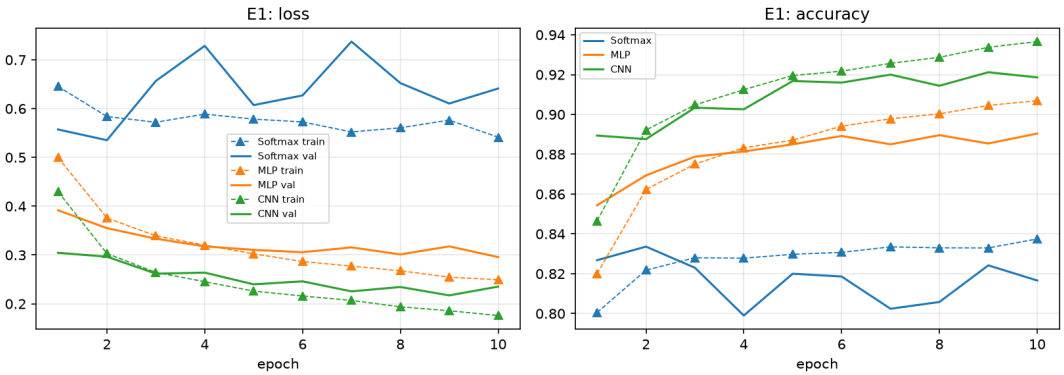


Figure 1: curves.png: Training and validation accuracy/loss curves for Softmax, MLP, and CNN (seed 0).

- **Fit / Overfit:** Softmax almost does not overfit (train ≈ val), but it underfits: loss remains high, and bias is large. MLP begins to separate train and val around epochs 5–6.

  CNN shows the clearest overfitting at the end: train acc (93.66%) vs. val acc (91.86%). Val loss bottoms out around epoch 9 and then increases at epoch 10. Future improvement directions: data augmentation (horizontal flip, slight translation) or early stopping. In this experiment, 10 epochs are kept constant for all three models for a fair comparison.

1.5 Error Analysis

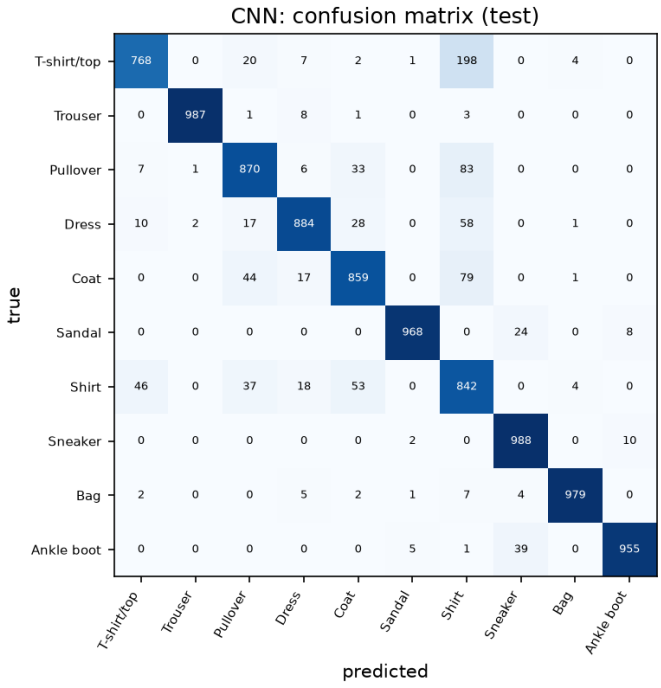


Figure 2: Confusion matrix for CNN (seed 0).

Accuracy by class:



Figure 3: Errors for CNN (seed 0).

Table 4: Accuracy by class for Softmax, MLP, and CNN (seed 0).

Class	**Softmax (%)**	**MLP (%)**	**CNN (%)**
T-shirt/top	86.0	85.1	76.8
Trouser	96.1	96.7	98.7
Pullover	81.7	83.6	87.0
Dress	71.0	92.0	88.4
Coat	71.9	80.1	85.9
Sandal	97.3	94.3	96.8
Shirt	46.3	66.1	84.2
Sneaker	81.6	95.6	98.8
Bag	93.6	97.1	97.9
Ankle boot	94.8	96.1	95.5

Most confused pairs (test set):

Table 5: Most confused pairs (test set).

Model	**Most Confused Pairs**
Softmax	Coat → Pullover: 194, Shirt → T-shirt/top: 190, Shirt → Pullover: 188
MLP	Shirt → T-shirt/top: 128, Coat → Pullover: 105, T-shirt/top → Shirt: 92
CNN	T-shirt/top → Shirt: 198, Pullover → Shirt: 83, Coat → Shirt: 79

- Errors are almost entirely concentrated in the upper-body clothing group (T-shirt, Shirt, Pullover, Coat). These four classes share the same overall shape, differing only in minor details (collar, buttons, sleeve length, zipper). At 28×28 resolution, these details occupy very few pixels.
- Shirt is the most difficult class and also the one where CNN shows the strongest improvement: 46.3% → 66.1% → 84.2% (+37.9 percentage points compared to Softmax). Collars and buttons are local features, which convolutional filters capture much better than a global linear projection.
- CNN shifts the errors rather than just reducing them: it pulls Shirt accuracy up but drops in T-shirt (86.0% → 76.8%) because it leans toward predicting Shirt for plain short-sleeved shirts. T-shirt → Shirt is the most confused pair for CNN (198 images). Overall, it is still a net gain because the increase in Shirt outweighs the decrease in T-shirt.

- Classes with distinct shapes (Trouser, Bag, Sandal, Ankle boot) already achieved high accuracy even with the linear model (around 93.6–97.3%). Sneaker, however, was low in Softmax (81.6%) and strongly improved by CNN (98.8%). Therefore, the majority of the improvement from MLP/CNN comes from the shirt/top group.
- Reviewing `errors_CNN.png`: the majority of the remaining error samples are images where the label itself is ambiguous (a plain long-sleeved shirt could be a Shirt / Pullover / Coat), so part of the error rate is due to the limits of the labeling and resolution, not just the model.

1.6 Conclusion

Test accuracy: Softmax (82.0%) $<$ MLP (88.7%) $<$ CNN (91.0%). The first performance jump comes from non-linearity, the second jump comes from the spatial inductive bias of convolutions, achieved with roughly 4 times fewer parameters than the MLP. The remaining errors are concentrated in visually similar clothing classes, especially Shirt / T-shirt.

2 E2 — Transformer: MSA and Tokenization

2.1 Overview

E2 trains a small Transformer encoder on the same Fashion-MNIST split as E1 (55,000 train / 5,000 validation / 10,000 test, seed 0, same normalization, no augmentation). Two things are compared:

(i) a **hand-written** Multi-Head Self-Attention (MSA) versus PyTorch's `nn.MultiheadAttention`, with the same tokenization;

(ii) **three tokenizations**: patches, rows, and a CNN stem.

Code: `E2/E2_Transformer.py`. Logs and figures: `E2/outputs/`.

2.2 Multi-Head Self-Attention

2.2.0.1 Hand-written MSA (`ScratchMSA`).

Built only from `nn.Linear`, `torch.einsum` and `softmax`. For tokens $X \in \mathbb{R}^{N \times D}$ with h heads and $d_h = D/h$:

$$
\begin{aligned}
[Q, K, V] &= XW_{qkv} + b_{qkv}, \quad W_{qkv} \in \mathbb{R}^{D \times 3D} \\
Q, K, V &\to \text{split into } h \text{ heads } Q_i, K_i, V_i \in \mathbb{R}^{N \times d_h} \\
\text{head}_i &= \text{softmax}\left(\frac{Q_i K_i^\top}{\sqrt{d_h}}\right) V_i \\
\text{MSA}(X) &= [\text{head}_1, \ldots, \text{head}_h]\, W_o + b_o
\end{aligned}
$$

One linear layer computes Q, K, V together. The result is reshaped to `[B, N, 3, h, d_h]` and permuted to three tensors of shape `[B, h, N, d_h]`. The attention scores `[B, h, N, N]` are computed with `einsum("bhid,bhjd->bhij")`, normalized by softmax over the keys, applied to V, and the heads are concatenated before the output projection W_o.

2.2.0.2 PyTorch MSA (`TorchMSA`).

`nn.MultiheadAttention(D, h, batch_first=True)`, called with `need_weights=False`.

2.2.0.3 Correctness check.

`ScratchMSA` stores its weights in the same layout as `nn.MultiheadAttention` (`in_proj_weight` $= [W_q; W_k; W_v]$, head i uses features $[i\,d_h, (i+1)\,d_h)$). Before training, the script copies the weights of a `ScratchMSA` into a `TorchMSA` and compares the outputs on random input: the maximum absolute difference is 7.5×10^{-8}, i.e. floating-point round-off. The script stops if the difference exceeds 10^{-5}.

2.2.0.4 Fair comparison.

For each tokenization, the model with PyTorch MSA is initialized with *exactly the same weights* as the model with hand-written MSA (state dict copied with renamed attention keys), and both see the same mini-batches in the same order (seeded `DataLoader`). The two runs therefore differ only in the attention implementation.

2.3 Tokenization

Every tokenizer maps an image `[B, 1, 28, 28]` to tokens `[B, N, D]`.

Table 6: The three tokenizations.

Tokenizer	**How tokens are formed**	**Tokens** N
Patch	Non-overlapping 4×4 patches, linear projection $16 \to D$ (implemented as `Conv2d` with kernel = stride = 4, which is the same as a linear layer on each flattened patch).	49
Row	Each image row (28 pixels) is one token, linear projection $28 \to D$.	28
CNN stem	The two conv blocks of the E1 CNN (Conv3×3-BN-ReLU-MaxPool, 32 and 64 channels) give a $64 \times 7 \times 7$ feature map; each of the 7×7 positions is a token, projected $64 \to D$.	49

2.4 Architecture and Training

- **Encoder:** learnable `[CLS]` token + learnable positional embedding ($N + 1$ positions), then 2 pre-norm Transformer blocks: $x \leftarrow x + \text{MSA}(\text{LN}(x))$, $x \leftarrow x + \text{FFN}(\text{LN}(x))$.
- **Size:** $D = 64$, $h = 4$ heads ($d_h = 16$), FFN $64 \to 128 \to 64$ with GELU, dropout 0.1 after attention and in the FFN.
- **Head:** LayerNorm $\to$ Linear($64 \to 10$) on the `[CLS]` output.
- **Training:** same hand-written loop as E1 (forward $\to$ cross-entropy $\to$ `backward()` $\to$ `optimizer.step()`); AdamW (lr 10^{-3}, weight decay 0.05), cosine learning-rate decay per step, gradient clipping at norm 1.0, batch 128, 10 epochs, seed 0. Unlike E1 (plain Adam), Transformers are known to train more stably with AdamW, a decaying learning rate and gradient clipping.
- **Hardware:** CPU, 10 threads. Time is measured per training epoch; inference throughput is measured on the full test set (batch 1000).

Table 7: Trainable parameters. The two MSA versions have exactly the same parameters.

Part	**Patch**	**Row**	**CNN stem**
Tokenizer	1,088	1,856	23,072
CLS + positional embedding	3,264	1,920	3,264
2 encoder blocks (33,472 each)	66,944	66,944	66,944
Final LayerNorm + head	778	778	778
Total	**72,074**	**71,498**	**94,058**

2.5 Results (test, seed 0)

Table 8: E2 results on the test set (seed 0, 10 epochs). Speed is measured separately with `-bench` on an idle CPU (median of 3): ms per training step (batch 128), estimated seconds per epoch (430 steps), and inference images per second (batch 1000). Accuracies in %.

Tokenizer	MSA	N	Params	Best val	Test acc	Test loss	ms/step	s/epoch	img/s
Patch 4×4	Hand-written	49	72,074	87.96	87.13	0.352	70	30	7,996
Patch 4×4	PyTorch	49	72,074	88.12	87.75	0.341	69	30	9,006
Row	Hand-written	28	71,498	89.48	88.83	0.310	43	18	15,951
Row	PyTorch	28	71,498	89.54	89.16	0.305	44	19	18,742
CNN stem	Hand-written	49	94,058	91.10	91.12	0.263	107	46	4,550
CNN stem	PyTorch	49	94,058	91.46	90.89	0.262	104	45	4,528
E1 CNN (reference)			50,282	92.12	91.00	0.255	–	–	–

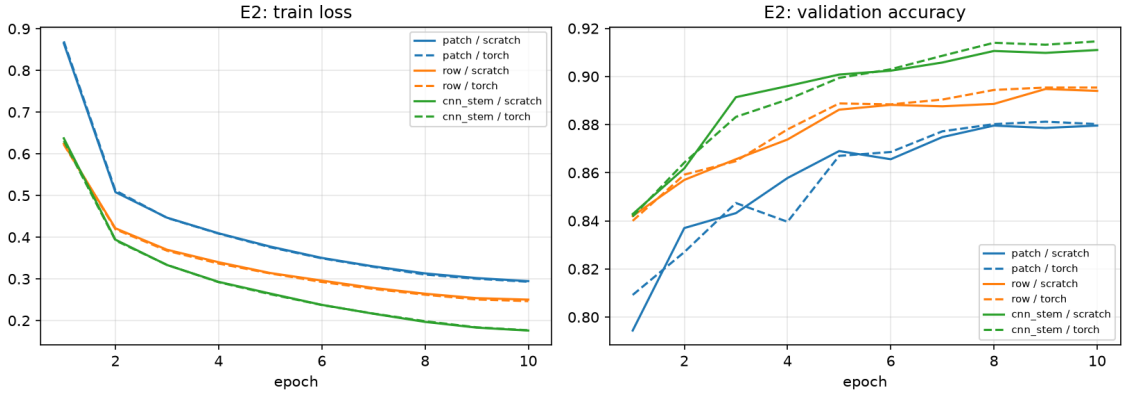


Figure 4: E2: training loss and validation accuracy of the six runs. Solid: hand-written MSA; dashed: PyTorch MSA; one colour per tokenizer.

2.6 Remarks

2.6.0.1 (i) Hand-written vs PyTorch MSA (same tokenization).

- **Same accuracy.** With identical initial weights, batches and dropout masks, the two versions end within 0.6 percentage points of each other, with no consistent winner: patch 87.13% vs 87.75%, row 88.83% vs 89.16%, CNN stem 91.12% vs 90.89%. The two implementations add floating-point numbers in a different order; the $\sim 10^{-7}$ differences already change the third decimal of the epoch-1 loss (patch: 0.8650 vs 0.8698) and are amplified over 4,300 updates. The curves of each pair overlap in Figure 4. Together with the 7.5×10^{-8} output check, this confirms that the hand-written MSA is correct.

- **Same training speed.** A training step takes the same time within 3% (patch 69.5 vs 69.1 ms, row 43.0 vs 43.9 ms): with $N \leq 50$ tokens and $D = 64$, most of the time goes into the Linear layers, LayerNorm and the backward pass, not into the attention product itself.

- **Faster inference with PyTorch.** With `need_weights=False`, `nn.MultiheadAttention` uses a fused scaled-dot-product kernel that does not materialize the $N \times N$ attention matrix: +13% images/s for patch (9,006 vs 7,996) and +17% for row (18,742 vs 15,951). For the CNN stem the convolutions dominate and both are equal (4,528 vs 4,550).

2.6.0.2 (ii) Tokenizations.

- **CNN stem is the most accurate** (91.12%), on par with the E1 CNN (91.00%). Its tokens already contain local features (edges, collars, sleeves) from convolutions with overlapping 10×10 receptive fields, and attention only has to combine them. It is also the most expensive: 94,058 parameters and 107 ms per step, about 2.5 times the row tokenizer.

- **Row beats patch** (88.83% vs 87.13%) with fewer tokens (28 vs 49) and 1.6 times faster steps. Each row token sees the full width of the garment (sleeve length, neckline, width of trousers), a strong cue on centred Fashion-MNIST images. A 4×4 patch is a flat projection of 16 pixels; the model has no built-in notion of neighbouring patches and must learn the 2D layout from the positional embedding alone.

- **Inductive bias matters more than attention on small data.** The two pure-attention tokenizers reach the level of the E1 MLP (88.86%) rather than the E1 CNN, with about a third of the MLP's parameters. With 55,000 images and 10 epochs, a Transformer without convolutional priors does not catch up with a small CNN; adding a CNN stem closes the gap. This is consistent with the ViT paper's observation that Transformers need large data or convolutional priors.

- **No overfitting yet.** The train–validation gap is small (patch 89.2% vs 88.0%, CNN stem 93.7% vs 91.1%) and validation loss is still decreasing at epoch 10 for all runs, so longer training would likely improve every model; 10 epochs were kept to match E1.

2.7 Error Analysis

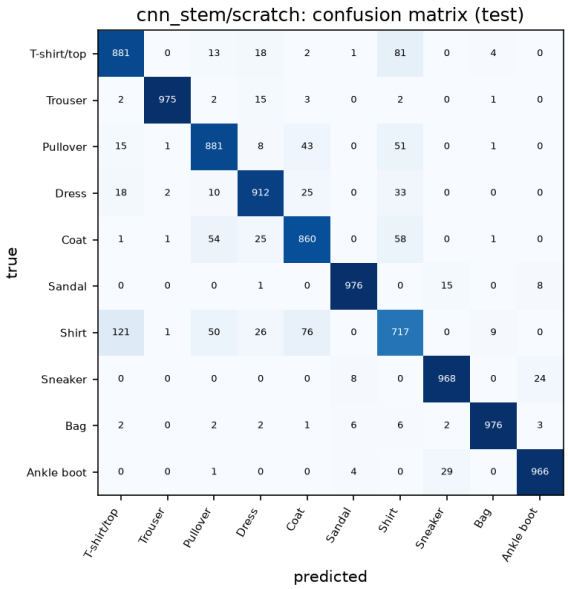


Figure 5: Confusion matrix of the best run (CNN stem, hand-written MSA, 91.12% test accuracy).

Table 9: Accuracy by class (%), hand-written MSA runs.

Class	Patch	Row	CNN stem
T-shirt/top	82.6	84.1	88.1
Trouser	96.7	97.9	97.5
Pullover	78.5	81.5	88.1
Dress	89.2	88.7	91.2
Coat	79.5	83.2	86.0
Sandal	96.3	96.8	97.6
Shirt	63.0	66.8	71.7
Sneaker	94.2	96.5	96.8
Bag	96.0	97.2	97.6
Ankle boot	95.3	95.6	96.6

Table 10: Most confused pairs on the test set (hand-written MSA runs).

Tokenizer	Most confused pairs (true → predicted: count)
Patch 4×4	Shirt → T-shirt/top: 142, T-shirt/top → Shirt: 112, Shirt → Pullover: 105
Row	Shirt → T-shirt/top: 136, T-shirt/top → Shirt: 107, Pullover → Coat: 85
CNN stem	Shirt → T-shirt/top: 121, T-shirt/top → Shirt: 81, Shirt → Coat: 76

2.8 Conclusion

The hand-written MSA matches `nn.MultiheadAttention` numerically (7.5×10^{-8}) and in training (accuracy within 0.6 points, same step time); PyTorch's fused kernel is only faster at inference (+13–17%). Among tokenizations, CNN stem (91.1%) > row (88.8%) > patch (87.1%): on a small dataset, how much local structure the tokens carry matters more than the attention implementation, and the Transformer needs a convolutional stem to match a plain CNN.

3 E3 — LSTM / GRU on Image Sequences

3.1 Training

Same dataset, same split (55k/5k/10k), and same training loop as E1 (imports directly from E1). Seed 0, 10 epochs, Adam with learning rate 2e-3, batch size 128. The data points below are directly comparable to those in E1.

3.2 Converting Image to Sequence

Image $(1 \times 28 \times 28) \rightarrow$ Sequence $(T \times D)$:

Table 11: Sequence tokenization methods

Name	Tokenization Method	T (Time Steps)	D (Step Dim.)
`row`	Each image row is a time step, scanned top-down.	28	28
`patch4`	Cut into non-overlapping 4×4 blocks, traversed row-major.	49	16

The `to_seq` function includes a self-check to verify the correct shape (T, D) and ensure no pixels are lost or duplicated during reshaping.

3.3 Architecture

1-layer LSTM/GRU, hidden size = 128, `batch_first`, taking the hidden state at the final time step $\rightarrow$ Linear($128 \rightarrow 10$). Both cell types share the exact same overall architecture, so parameter differences stem entirely from the cell formulas (LSTM has 4 gates, GRU has 3 $\rightarrow$ GRU has $\sim$25% fewer parameters). **Non-recurrent baselines:** MLP and CNN load checkpoints from E1 and are re-evaluated on this exact test set without retraining, ensuring perfectly identical conditions.

3.4 Results (test, seed 0)

Table 12: Performance comparison of sequential models vs baselines

Model	Sequence	Parameters	Best Val Acc	Test Acc	Train Time (CP
LSTM	`row` (28×28)	82,186	90.32%	89.55%	41 s
GRU	`row` (28×28)	61,962	89.82%	89.52%	72 s
LSTM	`patch4` (49×16)	76,042	87.46%	86.89%	69 s
GRU	`patch4` (49×16)	57,354	89.60%	88.45%	136 s
MLP (baseline)	—	203,530	89.04%	88.67%	10 s
CNN (baseline)	—	50,282	92.12%	91.00%	192 s

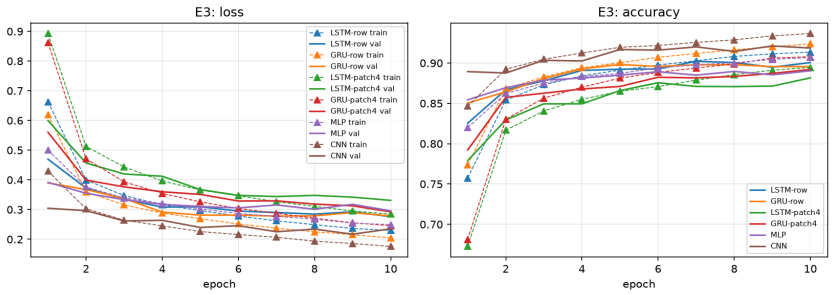


Figure 6: Loss and accuracy curves for E3 models.

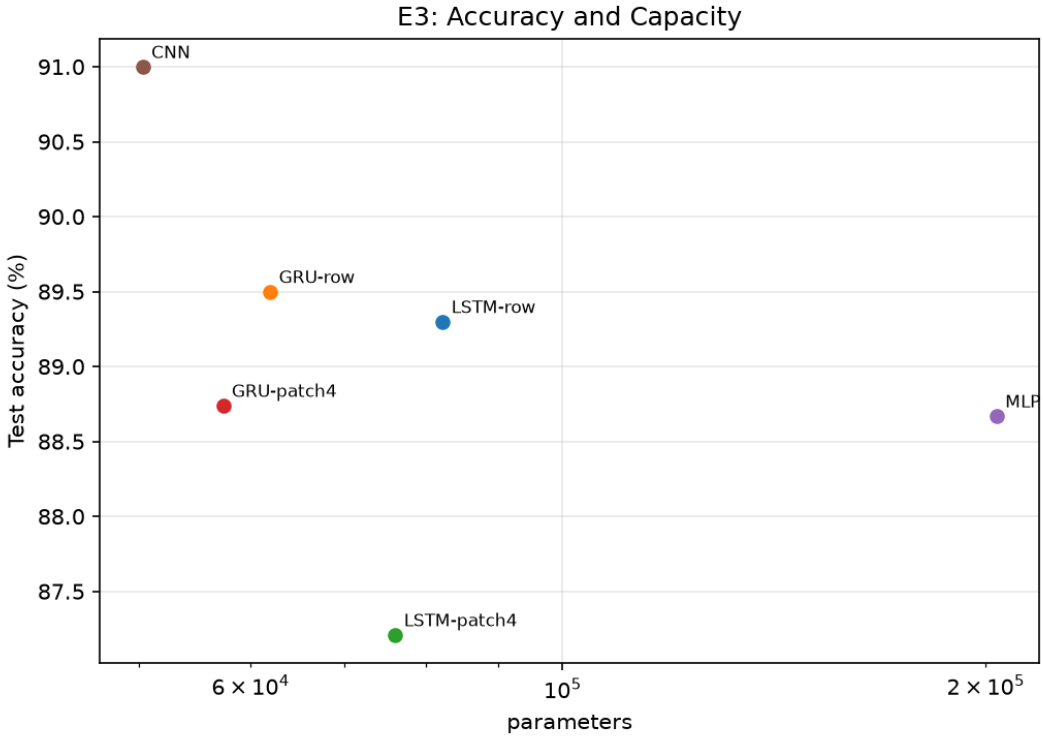


Figure 7: Accuracy and number of parameters for E3 models.

3.5 Remarks

- **LSTM vs. GRU:** On the `row` representation, both cells perform almost identically (89.55% vs. 89.52%), but GRU achieves this with 25% fewer parameters. On `patch4`, GRU performs better (+1.6%) because the sequence is longer (49 steps) and GRU's simpler gates are easier to optimize within 10 epochs. *Conclusion:* For a small problem like this, LSTM brings no benefit over GRU and only adds parameters.

- **Sequence Representation:** With LSTM, `row` (90.32% val) outperforms `patch4` (87.46% val) by nearly 3%. With GRU, both representations are nearly equal (89.52% vs. 89.60% test), even though `patch4` preserves 2D locality better. There are two reasons for this: (i) `row` has only 28 steps while `patch4` has 49, shorter sequences mean shorter gradient paths, reducing dilution when only using the final hidden state. (ii) Each `row` step sees a full horizontal line of 28 pixels, enough to

recognize if a shape "widens or narrows", whereas a 4×4 block contains very little information and requires aggregating across many steps. Furthermore, `patch4` is $\sim$1.8$\times$ slower due to having more time steps.

- **Compared to non-recurrent baselines:**
  - **RNN > MLP** (89.5% vs. 88.67%) with 2.5–3$\times$ fewer parameters. MLP treats 784 pixels as an unstructured vector, while RNN shares the same weights across all 28 rows and knows the top-to-bottom order of the rows. Shared weights + correct structural assumption $\rightarrow$ vastly superior parameter efficiency.
  - **CNN remains the best** (91.00%), outperforming LSTM-row by 1.45% while using fewer parameters. Images are not truly sequential data: pixel relationships are 2D and local, whereas RNNs are forced to read 1D top-to-bottom and compress the entire image into a 128-dimensional vector at the last step. 2D convolution aligns better with the nature of the data.
  - **Time:** RNN is faster than CNN on CPU (41–136 s vs. 192 s), but it cannot be parallelized over time, so this advantage will disappear on GPU or with longer sequences.
  - **Ability to capture temporal dependencies:** The RNN's advantage over MLP shows it successfully utilizes row ordering (information MLP lacks). However, the gap with CNN shows that "time" dependence here is merely a 1D slice of true 2D structure—it helps, but not as much as directly modeling 2D neighborhoods.

3.6 Conclusion

CNN (91.00%) > LSTM-row (89.55%) $\approx$ GRU-row (89.52%) > MLP (88.67%) > GRU-patch4 (88.45%) > LSTM-patch4 (86.89%). Representation by row is better than by 4×4 patches, GRU matches or beats LSTM with fewer parameters. RNNs are more parameter-efficient than MLPs due to weight sharing over time, but still lose to CNNs because images are fundamentally not 1D sequential data.

Academic Integrity Statement

In compliance with the course regulations, our group declares the use of AI tools as follows:

- **Purpose of use:** AI was utilized as a learning support tool, specifically for:
  - Suggesting and analyzing the pros and cons to help the group decide on selecting the Fashion-MNIST dataset instead of MNIST or CIFAR-10.
  - Assisting in researching and explaining theoretical concepts (e.g., the mechanisms of Softmax, Cross-Entropy, and sequence structures in LSTM/GRU).
  - Helping to refine the grammar and phrasing within the report.